SMARTGYM DEVELOPER GUIDE

Development Environment

The firmware is a PlatformIO Arduino project for an ESP32-S3 VIEWE display board.

Required environment:

```text
BOARD_VIEWE_UEDX80480070E_WB_A
```

Build:

```powershell
python -m platformio run --environment BOARD_VIEWE_UEDX80480070E_WB_A
```

For local hardware builds, provide private Wi-Fi and Firebase values through PlatformIO build flags or a local untracked config. Do not commit real SSIDs, passwords, Firebase tokens, or private project configuration.

Upload and monitor:

```powershell
python -m platformio run --target upload --target monitor --environment BOARD_VIEWE_UEDX80480070E_WB_A
```

Repository Layout

The firmware remains at repository root to preserve existing PlatformIO build behavior:

- `src/`: application code.
- `lib/`: firmware modules.
- `boards/`: board configuration.
- `dashboard/`: React dashboard.
- `weight_detection/`: standalone VL53L0X prototype.
- `docs/`: project documentation.
- `hardware/`: wiring notes.
- `screenshots/`: demo images.

Main Firmware Architecture

‘SmartGymTouchApp‘ is the application orchestrator. It owns the UI state, training state, calibration flow, user sync, Firebase queueing, and memory guards.

Main state categories:

- Idle/Ready.
- Calibration.
- Training.
- Summary.

Main UI screen modes include:

- Main.
- Summary.
- Idle.
- Calibration.
- Calibration gate.
- Debug.
- Profile.

Main Modules

SmartGymTouchApp

Coordinates:

- LVGL UI creation and refresh.
- RFID scan flow.
- User profile/calibration sync.
- ROM normalization.
- Rep handling.
- Calibration state machine.
- Session start/finish.
- Firebase upload scheduling.
- Memory-aware upload chunking.

SensorManager

Reads the analog motion sensor and converts it into a filtered ROM percentage. The firmware configures sampling at 5 ms, approximately 200 Hz.

RepDetector

Uses a ROM-based state machine:

- Bottom.

- Ascending.
- Top.
- Descending.

It computes rep duration, ROM range, velocities, warnings, and invalid flags.

SessionRecorder

Stores the active session, sets, reps, per-rep load, timing, ROM, quality metrics, and JSON payloads for Firebase.

Current limits:

- Maximum sets per session: 8.
- Maximum reps per session: 80.

FirebaseService

Handles Firebase RTDB paths, JSON construction, reads, writes, and FirebaseClient queued transport.

MachineRegistry

Stores the embedded machine catalog and motion target templates for goals:

- ‘hypertrophy‘
- ‘strength‘
- ‘endurance‘
- ‘test‘

The ‘test‘ goal is retained for internal/debug round-trip validation.

UserRegistry

Stores local user profiles and user-machine calibration data.

Current limits:

- Maximum local profiles: 8.
- Maximum machine calibrations per profile: 8.

LocalPersistenceStore

Uses NVS/Preferences for:

- Local user cache.
- Local session cache.
- Pending upload queue.

The upload queue is used for retry/offline resilience.

RfidService

Reads MFRC522 cards and formats UIDs as uppercase hex groups separated by ‘-‘.

UI Architecture

The firmware uses LVGL. UI refresh is intentionally split into different rates to reduce CPU and memory pressure:

- Main app tick: 4 ms.
- Chart refresh: about 16 ms.
- Slow UI refresh: about 140 ms.
- Debug UI refresh: about 160 ms.

Screen-specific LVGL objects must be reset to ‘nullptr‘ when their parent screen/modal is deleted. This prevents stale pointer crashes.

Calibration Architecture

Calibration is user-specific and machine-type-specific. The flow includes:

- Start load confirmation.
- Physical pin-load controls.
- Rep collection.
- Set analysis.
- Optional next load.
- Result and save.

Calibration records valid/rejected reps, ROM, velocity, load, confidence, action, and recommendation data. Calibration is saved under:

‘‘‘text
calibrations/{uid}/{machineTypeId}
‘‘‘

Recommendation Logic

Recommendation is separate from physical pin load.

- Pin load is machine-level physical state.
- Recommendation is user-specific advice.
- Recommendation must not automatically overwrite pin load.

The firmware resolves recommendation from the latest valid user-machine data, including session summary or calibration data where available.

Firebase Sync Pipeline

Session finish does not perform heavy Firebase work directly. It schedules a queued upload.

Upload phases:

1. Session root/core.
2. Day metadata.
3. Day timeline.
4. Day summary.
5. Week summary.
6. Representative reps.
7. Set details.
8. Rep sets.

The firmware preserves final Firebase shape while splitting payloads when memory is low.

Queue and Chunking Behavior

The firmware uses memory modes:

- Normal.
- Constrained.
- Critical.

RepSet upload limits:

- Normal: target 1900 bytes, hard max 2100 bytes, up to 6 reps per patch.
- Constrained: target 900 bytes, hard max 1100 bytes, up to 3 reps per patch.
- Critical: target 650 bytes, hard max 850 bytes, up to 2 reps per patch.

The final Firebase structure remains:

‘‘‘text
repSets/set1/reps/0
repSets/set1/reps/1
‘‘‘

No chunk nodes are added to the final schema.

Adding a New Machine

1. Add a machine seed in ‘MachineRegistry‘.
2. Define machine ID, machine type ID, display name, category, muscles, stroke length, min/max/increment, and default calibration weight.
3. Confirm goal templates and timing targets.
4. Rebuild firmware.
5. Verify Firebase machine config compatibility if cloud overrides are used.

Editing Goals

Public goals:

- ‘hypertrophy‘
- ‘strength‘
- ‘endurance‘

Internal/debug goal:

- ‘test‘

Legacy compatibility:

- ‘general‘, normalized for new runtime behavior.

Do not remove ‘test‘; it is used for device/Firebase testing.

Common Serial Log Tags

- ‘[BOOT]‘: startup.
- ‘[BOOTMEM]‘: memory checkpoints.
- ‘[RFID]‘: card scans and user load.
- ‘[USER_SYNC]‘: profile/calibration/recommendation load.
- ‘[UI_LOADING]‘: loading popup.
- ‘[WEIGHT]‘: pin load and recommendation decisions.
- ‘[SESSION]‘: session start, reps, sets, finish.
- ‘[CAL]‘: calibration logic.
- ‘[CAL_UI]‘: calibration screen updates.
- ‘[SYNC]‘: upload and queue behavior.
- ‘[SYNCGATE]‘: cloud sync scheduling.
- ‘[CloudWrite]‘: Firebase writes.
- ‘[GRAPH]‘: graph/timing display.

Memory Considerations

Firebase/TLS can be memory-sensitive on ESP32. The firmware:

- Frees optional UI before training/upload.
- Uses NVS queueing.
- Limits payload sizes.
- Splits repSet patches under low memory.
- Avoids upload workers.
- Avoids direct heavy upload inside ‘finishTraining‘.

Known Limitations

- Physical pin load is manual in the main firmware.
- Weight detection is a separate prototype.
- Firebase dashboard config is environment-based; local ‘.env‘ files must remain untracked.
- Automated test coverage is limited.